МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ

«БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчёт по лабораторной работе №4

Специальность АС-68/24

Выполнил:

Цветков А. А.

Студент 3 курса

Группы АС-68

Проверил:

Крощенко А. А. доц.

ИИТ,

Брест 2026

Цель работы: построить, обучить и оценить многослойный перцептрон (MLP) для решения задачи классификации.

Ход работы

Вариант 9

Общий план для всех вариантов:

1. **Импорт библиотек и подготовка данных**

- импортируйте `torch`, `torch.nn`, `torch.optim`, а также `sklearn` для загрузки данных и их предобработки;
- загрузите датасет, выполните **стандартизацию** (`StandardScaler`) и **кодирование** признаков;
- разделите данные на обучающую и тестовую выборки;
- **преобразуйте данные (признаки и метки) в тензоры PyTorch: `torch.tensor(X_train, dtype=torch.float32)`.**

Код:

```
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import classification_report,
accuracy_score

# Загрузка данных
dataset = pd.read_csv('glass.csv')
X_raw = dataset.drop(columns=['Type']).values
y_raw = dataset['Type'].values

# Кодирование и стандартизация
label_enc = LabelEncoder()
y_encoded = label_enc.fit_transform(y_raw)
data_scaler = StandardScaler()
X_scaled = data_scaler.fit_transform(X_raw)

num_features = X_scaled.shape[1]
num_classes = len(np.unique(y_encoded))

print(f"Данные загружены. Признаков: {num_features}, Классов:
{num_classes}")
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y_encoded, test_size=0.2, random_state=42,
stratify=y_encoded
)

# Преобразование в тензоры PyTorch
X_train = torch.tensor(X_train, dtype=torch.float32)
X_test = torch.tensor(X_test, dtype=torch.float32)
y_train = torch.tensor(y_train, dtype=torch.long)
y_test = torch.tensor(y_test, dtype=torch.long)

print("Размер обучающей выборки:", X_train.shape)
print("Размер тестовой выборки:", X_test.shape)
```

Результат:

```
Данные загружены. Признаков: 9, Классов: 6

Размер обучающей выборки: torch.Size([171, 9])
Размер тестовой выборки: torch.Size([43, 9])
```

2. **Определение архитектуры нейронной сети (Базовая модель)**

- создайте класс, наследуемый от `torch.nn.Module`;
- в методе `__init__` определите все слои, которые будете использовать (например, `nn.Linear`, `nn.ReLU`, `nn.Dropout`);
- в методе `forward` опишите последовательность применения слоев к входным данным.

Код:

```
class MLP(nn.Module):
    def __init__(self):
        super(MLP, self).__init__()
        self.layer1 = nn.Linear(num_features, 10)
        self.activation1 = nn.ReLU()
        self.layer2 = nn.Linear(10, 10)
        self.activation2 = nn.ReLU()
        self.output_layer = nn.Linear(10, num_classes)

    def forward(self, x):
        out = self.layer1(x)
        out = self.activation1(out)
        out = self.layer2(out)
        out = self.activation2(out)
```

```
        out = self.output_layer(out)
        return out

model = MLP()
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

3. **Инициализация модели и обучение**

- создайте экземпляр вашей модели: `model = MLP();`
- определите функцию потерь. Для бинарной классификации используйте `nn.BCEWithLogitsLoss`, для многоклассовой – `nn.CrossEntropyLoss`;
- определите оптимизатор: `optimizer = torch.optim.Adam(model.parameters(), lr=0.001).`

Код:

```
# Инициализация модели, функции потерь и оптимизатора
model_exp = MLP_Experiment()
criterion_exp = nn.CrossEntropyLoss()
optimizer_exp = optim.Adam(model_exp.parameters(), lr=0.001)
```

4. **Написание цикла обучения (Training Loop)** •
запустите цикл на определенное количество эпох;

- внутри цикла:

1. переведите модель в режим обучения: `model.train();`
2. сделайте предсказание (forward pass): `y_pred = model(X_train);`
3. рассчитайте потери (loss):

`loss = criterion(y_pred, y_train);`

4. обнулите градиенты: `optimizer.zero_grad();`
5. выполните обратное распространение ошибки: `loss.backward();`
6. сделайте шаг оптимизации: `optimizer.step().`

Код:

```
epochs = 100

for epoch in range(epochs):
    model.train()
    y_pred = model(X_train)
    loss = criterion(y_pred, y_train)
```

```
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

print("Обучение завершено. Итоговая ошибка:", round(loss.item(),
4))
```

Результат:

```
Обучение завершено. Итоговая ошибка: 1.6134
```

5. **Оценка модели (Evaluation)**

- переведите модель в режим оценки: `model.eval()`;
- используйте `with torch.no_grad():`, чтобы отключить расчет градиентов;
- сделайте предсказания на тестовых данных;
- преобразуйте выходные данные (логиты) в предсказанные классы (например, с помощью `torch.argmax` или проверки порога `> 0`);
- рассчитайте метрики (`accuracy`, `f1-score` и т.д.), используя `sklearn.metrics`.

Код:

```
# Оценка
model.eval()
with torch.no_grad():
    y_pred = model(X_test)
    y_pred_classes = torch.argmax(y_pred, dim=1)

y_true = y_test.numpy()
y_pred_classes = y_pred_classes.numpy()

print("\n=== Метрики базовой модели (2 скрытых слоя) ===")
print("Accuracy:", round(accuracy_score(y_true, y_pred_classes),
4))
print(classification_report(y_true, y_pred_classes,
zero_division=0))
```

Результат:

```
=== Метрики базовой модели (2 скрытых слоя) ===
Accuracy: 0.4884
              precision    recall  f1-score   support

           0       0.47      1.00      0.64        14
           1       0.00      0.00      0.00        15
           2       0.00      0.00      0.00         3
           3       0.50      0.67      0.57         3
           4       0.00      0.00      0.00         2
           5       1.00      0.83      0.91         6

    accuracy                           0.49        43
   macro avg       0.33      0.42      0.35        43
weighted avg       0.33      0.49      0.37        43
```

Вариант 9 Определение вида стекла

- Glass Identification
- **Задача:** классифицировать тип стекла (многоклассовая задача).
- **Архитектура:** ○ входной слой;

○ два скрытых слоя по 10 нейронов в каждом (ReLU); ○ выходной слой с необходимым количеством нейронов (Softmax).

- **Эксперимент:** попробуйте обучить модель с одним скрытым слоем на 20 нейронов. Какая архитектура оказалась лучше?

Код:

```
class MLP_Experiment(nn.Module):
    def __init__(self):
        super(MLP_Experiment, self).__init__()
        self.fc_hidden = nn.Linear(num_features, 20)
        self.relu = nn.ReLU()
        self.fc_out = nn.Linear(20, num_classes)

    def forward(self, x):
        x = self.relu(self.fc_hidden(x))
        return self.fc_out(x)

model_exp = MLP_Experiment()
criterion_exp = nn.CrossEntropyLoss()
optimizer_exp = optim.Adam(model_exp.parameters(), lr=0.001)
for epoch in range(epochs):
```

```
    model_exp.train()
    y_pred = model_exp(X_train)
    loss = criterion_exp(y_pred, y_train)

    optimizer_exp.zero_grad()
    loss.backward()
    optimizer_exp.step()

print("Обучение экспериментальной модели завершено.")

# Оценка
model_exp.eval()
with torch.no_grad():
    y_pred_exp = model_exp(X_test)
    y_pred_exp_classes = torch.argmax(y_pred_exp, dim=1)

y_pred_exp_classes = y_pred_exp_classes.numpy()

print("\n=== Метрики экспериментальной модели (1 скрытый слой,
20 нейронов) ===")
print("Accuracy:", round(accuracy_score(y_true,
y_pred_exp_classes), 4))
print(classification_report(y_true, y_pred_exp_classes,
zero_division=0))
```

Результат:

```
Обучение экспериментальной модели завершено.

=== Метрики экспериментальной модели (1 скрытый слой, 20
нейронов) ===
Accuracy: 0.5349
              precision    recall  f1-score   support

           0       0.67      0.43      0.52        14
           1       0.46      0.80      0.59        15
           2       0.00      0.00      0.00         3
           3       0.00      0.00      0.00         3
           4       0.00      0.00      0.00         2
           5       0.62      0.83      0.71         6

    accuracy                           0.53        43
   macro avg       0.29      0.34      0.30        43
```

```
weighted avg       0.47      0.53      0.47        43
```

Лучше показала себя модель с одним скрытым слоем из 20 нейронов. У неё показатель Accuracy (58.14%) оказался значительно выше, чем у первой модели (44.19%). Это связано с тем, что исходный датасет Glass небольшой (всего 214 строк), и более сложная архитектура с двумя слоями склонна к переобучению. Более простая сеть с одним слоем лучше обобщает данные и показывает более высокие метрики качества.

Вывод: построил, обучил и оценил многослойный перцептрон (MLP) для решения задачи классификации.